



INDIA.RUNS

Build what next India runs on

Track 1

AI Systems & Workflow Innovation Challenge

Redrob Context & Ideathon Scope

Redrob AI is building an AI-native ecosystem spanning hiring, productivity, communication, discovery, workflows, recommendations, automation, and intelligent user experiences.

Participants are encouraged to think beyond standalone applications and explore how their ideas can:

Extend existing
Redrob
capabilities

Improve
experiences for
Redrob users

Introduce
new AI-
native
workflows

Strengthen
ecosystem
engagement and
network effects

Important

Participants are not expected to build a completely new platform from scratch.

Strong submissions will demonstrate:

How the idea leverages existing Redrob capabilities

What new capability, workflow, or value layer is introduced

How the idea strengthens the overall Redrob ecosystem

Why Redrob is uniquely positioned to enable this experience

INDIA.RUNS

Build what next India runs on

Track 01

AI Systems & Workflow Innovation Challenge

Team Name : Disha and Sethu

Problem Statement : Redrob's candidate discovery relies on keyword search and opaque ranking — missing qualified candidates who describe their work differently, rewarding keyword-stuffers, and providing zero explanation for why any candidate ranked where they did.

Team Members : Chalasani Sreenivasulu

Problem Definition

Questions

- Redrob's candidate ranking is bottlenecked by keyword search and black-box ML — recruiters cannot audit decisions, qualified candidates get filtered out because they described the same skills differently, and keyword-stuffers consistently outrank genuine talent.
- Recruiters waste hours reviewing a poorly-ranked shortlist. Strong candidates get buried because they wrote "Ranking Systems" instead of "Learning to Rank." Both sides lose trust in Redrob as a reliable matching platform.
- Keyword search misses semantic equivalents and ignores behavioral signals entirely. Black-box ML may rank correctly on average but cannot explain why — making it impossible to audit, correct, or legally defend hiring decisions, and impossible to improve without ground-truth labels.

Opportunity & Vision

Questions

- India's tech hiring market processes millions of applications annually. A 10% precision improvement at Redrob's scale means thousands of better matches per month.
Explainability unlocks enterprise and regulated-industry clients who need audit trails — a market segment currently inaccessible to opaque-ranking platforms.
- Every recruiter query returns a ranked shortlist with a plain-English explanation per candidate. Every candidate knows exactly why they ranked where they did. The system autonomously improves from every recruiter shortlist and reject action — no manual retraining, no data science team required.

Solution Overview

Questions

- MatchMind — a multi-agent autonomous recruiting intelligence system. Five specialized agents (JD Parser, Candidate Ranker, Explanation Generator, Fraud Detector, Feedback Learner) coordinate through an Orchestrator to convert any recruiter query into a ranked, explained, continuously-improving shortlist.
- Every step runs autonomously — JD parsing, candidate scoring, explanation generation, fraud detection, and model retraining all happen without human intervention. Recruiters set intent (paste a JD); the agent network does the work. AI is not a helper inside a human workflow; it IS the workflow.
- Directly extends Redrob's existing candidate database, behavioral signals (redrob_signals), search/discovery surface, and shortlist/communication workflows.

User Journey / Workflow Diagram

Questions

- Recruiter pastes a JD into Redrob search (no form-filling, no keyword tagging). MatchMind returns a ranked list within seconds. Each candidate card shows a 1-sentence explanation. Recruiter clicks Shortlist or Reject — that action silently feeds the Feedback Agent. No new UI to learn.

Mandatory Visual

- JD text → JD Parser Agent extracts structured requirements JSON → Ranking Agent scores full candidate pool (semantic + feature signals) → Fraud Agent caps honeypots → [Recruiter enters JD] → [JD Parser Agent] → [Requirements JSON] → [Ranking Agent] → Explanation Agent generates per-candidate reasoning → ranked list returned → recruiter [Fraud Agent] → [Scored + Explained List] → [Recruiter: Shortlist / Reject] → [Feedback action] → Feedback Agent updates model async. Agent] → [Model Update] ↻ (continuous loop)
- Search results page (ranked list replaces keyword results), candidate profile page (explanation panel), shortlist/reject buttons (feedback capture), and recruiter dashboard

AI Logic & Decision Flow

Questions

- Five intervention points: (1) JD parsing — LLM extracts structured requirements from raw JD text; (2) Semantic matching — embedding model computes candidate-JD similarity; (3) Scoring — weighted formula combining 8 feature dimensions; (4) Fraud detection — structural rule agent caps honeypot profiles; (5) Feedback learning — online model

Mandatory Visual

update from every recruiter action.

- $\text{Score} = (0.32 \times \text{skill_match} + 0.22 \times \text{semantic_similarity} + 0.16 \times \text{experience_fit} + 0.12 \times \text{product_company} + 0.10 \times \text{shipped_evidence} + 0.08 \times \text{location}) \times \text{archetype_gate} \times \text{Embeddings} + \text{Scorer}$
- [Orchestrator] → [JD Parser Agent: LLM] → [Requirements JSON] | [Ranking Agent: Embeddings + Scorer] ↔ [Fraud Agent: Rules] → [Explanation Agent: Template+Facts] → disqualifier multiplier × behavioral modifier. Every weight is JD-justified and auditable. [API Response] || async: [Recruiter Event] → [Feedback Agent] → [Model Store Update] Honeypot cap overrides all.
- Orchestrator routes JD to Parser Agent (LLM) → parsed JSON feeds Ranking Agent

System Architecture

Questions

- Six components: (1) Orchestrator API — routes requests and coordinates agents; (2) JD Parser Agent — LLM extracts structured requirements from any JD text; (3) Candidate Intelligence Store — vector DB (embeddings) + structured DB (features + behavioral signals); (4) Ranking Engine — semantic scorer; (5) Explanation Generator; (6) Fraud/Integrity Agent; (7) Feedback Learning Service — async model update.

- All agents communicate via REST through the Orchestrator. Recruiter actions publish to [Redrob Search UI] → [Orchestrator API] → [JD Parser Agent (LLM)] + [Ranking Engine an event bus (Kafka/Redis Streams). Feedback Learning Service consumes events async (Vector DB + Scorer)] + [Fraud Agent (Rules)] + [Explanation Generator] → [Ranked Results to UI] | async: [Shortlist/Reject Event Bus] → [Feedback Learning Service] → another directly — all coordination through Orchestrator, enabling independent scaling and deployment of each agent.

Data, Context & Intelligence Layer

Questions

- Four data streams: (1) Candidate structured profiles — current_title, skills (name/proficiency/duration), career_history (company/title/description/duration); (2) Redrob behavioral signals — last_active_date, recruiter_response_rate, interview_completion_rate, notice_period_days, saved_by_recruiters_30d, open_to_work_flag; (3) Recruiter JD text — unstructured input, parsed at query time; (4) Recruiter feedback events — shortlist/reject actions per candidate.

Mandatory Visual

- [Candidate DB] → [Offline Embedder] → [Vector Store (FAISS/Qdrant)] | [Redrob Signals API] → [Feature Cache] | [Recruiter JD] → [JD Parser] → [JD Embedding] → [Ranking Engine] stored in FAISS/Qdrant; retrieved at query time by cosine similarity against the JD embedding. Structured feature scores (skill_match, experience_fit, product_company_score, etc.) cached per candidate and refreshed on profile update.
- [Events] → [Feedback Store] → [Model Weight Update]

Scalability & Technical Feasibility

Questions

- Three phases: Phase 1 (Month 1-2) — deploy rule-based ranker (already built, validated on 100K candidates at 50s runtime, zero dependencies); Phase 2 (Month 3-5) — add LLM JD Parser Agent + BGE embedding upgrade + Explanation Generator; Phase 3 (Month 6-9) — deploy Feedback Learning Service + async event bus + online model update loop. Each phase independently shippable.
- Rule-based feature scoring scales linearly ($O(n)$ candidates, no model inference at rank time). Embedding similarity with FAISS ANN indexing scales to tens of millions of candidates with sub-second retrieval. Feedback Learning runs async, never on the critical path. JD Parser Agent (LLM) called once per JD, not per candidate — cost is constant regardless of pool size. Each agent independently horizontally scalable.

Redrob Ecosystem Integration

Questions

- Candidate database (profiles, skills, career history), redrob_signals behavioral layer (23 live engagement signals per candidate), search/discovery UI (result list, candidate profile page), shortlist/reject workflow (feedback capture), recruiter session management (JD input), and communication workflows (rejection messaging can be auto-generated from explanation text).

Mandatory Visual

- Four net-new capabilities Redrob does not have today: (1) Autonomous multi-agent JD parsing — any JD text → structured requirements, no human tagging; (2) Per-candidate grounded explanation — every ranked result ships with a 1-sentence auditable reason; (3) Structural fraud/honeypot detection — impossible profiles automatically excluded; (4) Self-improving feedback loop — model improves from recruiter actions autonomously.

Impact & Success Metrics

Questions

- Primary metric: NDCG@10 improvement vs keyword baseline (target: +15-25% in offline eval). Secondary: recruiter time-to-shortlist reduction (target: -30%), shortlist-to-interview conversion rate (target: +20%), honeypot/fraud detection rate (target: 100% of structurally-impossible profiles excluded), and recruiter satisfaction NPS (tracked quarterly).
- A/B test: random split of new JDs routes to MatchMind or existing Redrob search, track recruiter shortlist conversion downstream. Offline evaluation: NDCG/MAP/P@10 vs ground-truth labels derived from historical shortlist data. Explanation quality audit: random 50-row sample scored monthly on the spec's 6 criteria (specificity, JD connection, honesty, no hallucination, variation, rank-consistency).

Future Roadmap

Questions

- Year 1: MatchMind live inside Redrob search — all recruiters get ranked + explained results by default. Year 2: candidate-facing job recommendations (same engine, inverted) + proactive talent pool alerts (Orchestrator runs matching autonomously on schedule). Year 3: cross-company skill demand forecasting (“which skills are being searched most this quarter”) + candidate career-path suggestions driven by gap analysis.
- Real-time market demand signals (which skills are being queried most — data only Redrob has at scale), enterprise audit dashboard for HR compliance (every hiring decision with a timestamped, grounded explanation on record), cross-language JD support (multilingual JD Parser Agent), and multi-modal candidate signals (GitHub activity, portfolio links, certifications beyond current redrob_signals).



INDIA.RUNS

Build what next India runs on

THANK YOU